DEEPSEEK · INFERENCE SYSTEM · 2026

DeepInfer

深度解析：为一个模型，重写一套引擎

从 vLLM 的骨架出发，看 C++ 调度器、ZOS 流水线、
YoCo 截断与 DSpark 投机如何咬合。

repo deepseek-ai/DeepInfer commit 9bd9cf2 license Apache-2.0 55k py · 5.9k cpp

THESIS

它不是“又一个 vLLM”，
而是 vLLM 被拆开、为一个模型重新焊死的样子。

55k
行 Python / 270 个文件

5.9k
行 C++ / 33 个文件

1
个模型：V4.1-Flash

代码量统计：`find . -name '*.py' -exec cat {} + | wc -l`

BOUNDARIES

先排除四个错误预期

✕ 通用框架

只认 V4.1-Flash 的 HF checkpoint；换模型要改模型代码 + kernel，不是改配置。

✕ 多代硬件

只支持 SM100 / Ascend 950。H100/A100 在 `get_bf16_tflops_limit()` 直接抛错。

✕ 在线服务

没有 OpenAI server，没有 AsyncLLMEngine；`entrypoints/` 只有离线 LLM。

✕ 稳定 API

v0.1.0、4 个 commit、154 个 PR；接口随时会变，学习请 pin commit。

NUMBERS

三个反直觉的数字

01

42%

Python 代码里是 TileLang kernel（23,331 / 54,929 行）。真正的框架逻辑只有约 3 万行。

02

0

全仓库 `tensor_parallel` / `tp_size` 出现次数。并行只有 EP / DP / CP。

03

64

个 `DEEPINFER_*` 环境变量。调度、缓存、kernel、调试行为大量靠 env 切换。

LINEAGE

三种基因，一次重写

vLLM

引擎骨架

LLMEngine / Worker / ModelRunner / CacheEngine / Sequence 的分层与命名。

文件头：`Derived from vLLM`

SGLang

异步调度

ZOS 的四段流水线。Q&A 原文：“Following SGLang, NanoFlow”。

TensorRT-LLM

图与融合

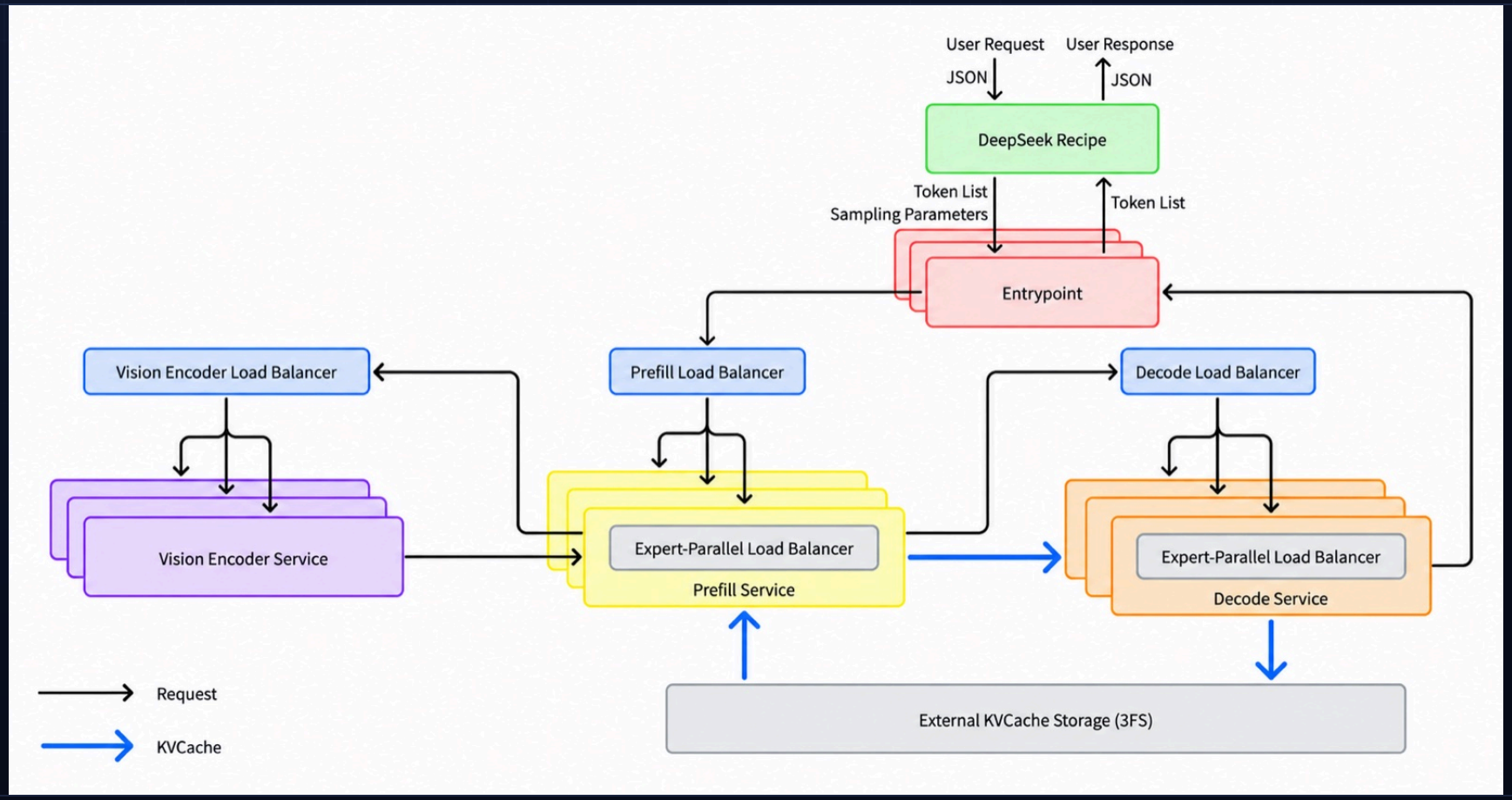
CUDA Graph、kernel 融合、profile 驱动的调优思路。



最不像 vLLM 的地方：调度器从 Python 搬进了 C++（`csrc/engine/`）。

SYSTEM OVERVIEW

开源的是“引擎”，不是“系统”



Entrypoint ×3 · VE/Prefill/Decode LB · EPLB · 3FS KV Store：中间一圈在本仓库；外圈（KV 存储、负载均衡、扩缩容、Recipe）未开源。

ENGINE TYPES

一个类，三种形态

NORMAL

P + D 合一

既做 prefill 又做 decode。开发调试、小规模部署。

`step` 里投票决定这一步做哪种

PROMPT

只做 prefill

算完把 prompt 发给 completion；不做 decode。

`assert prompt_run`

COMPLETION

只做 decode

EP64–128 的 decode 集群；不接收新 prompt。

必须 `stop_preempted_request=true`

```
// csrc/engine/scheduler.hpp — 三种形态共用一套调度代码，靠钩子分化
case EngineType::PROMPT:     return _schedule_prefill(acquire);
case EngineType::COMPLETION: return _schedule_decode(acquire);
```

CODE MAP

5.5 万行怎么分布



读的顺序应该反过来：先 engine/worker 的 5k 行骨架，再按需下潜到 models 和 kernel。

REQUEST LIFECYCLE

一个请求的七个阶段

tokenize → add_request → schedule() → execute_model → sample → post_process → RequestOutput

阶段	输入	输出	关键点
tokenize	字符串	`np.int32[]`	引擎不收字符串；VL 会改写成 image token
add_request	tokens + params	`Sequence`	SO 编译异步提交；非法 token 直接丢
schedule	队列状态	`SchedulerOutputs`	C++；每步选 batch、切 chunk
execute	`SequenceMetadata`	hidden / logits	只读快照，避免执行期被改
sample	logits	`SamplerOutput`	fused kernel；含投机验证
post	SamplerOutput	指针数组	对象池 + 延后释放

LLMENGINE.STEP()

每步的四段流水线，以及谁在等谁

CPU

schedule

C++ 选 batch；克隆/清零 state；`post_schedule` 钩子。

GPU

forward

主模型；decode 走 CUDA Graph；DFlash 走独立 graph。

GPU

sample

fused_sample：温度、top-k/p、log-softmax、采样、SO mask。

CPU

post-process

推进序列状态、判断结束、构造输出、统计。

```
# llm_engine.py:860 — 输出是指针数组，不是 Python 对象列表
request_outputs = self._execute_model_async_step(seq_metadata_list, scheduler_outputs)
self.deferred_release.collect_request_outputs(request_outputs)  # 下一步才释放
```

把调度器搬进 C++：一次明确的取舍

Python 调度器的成本

- 每个 seq 的字段访问都走对象协议
- 高频小对象触发 GC 抖动
- decode 步只有几毫秒，CPU 占比被放大

C++ 调度器的代价

- 调试跨语言，Python 断点看不到内部
- 改调度要重编译
- pybind 接口手写，字段变更要同步 .pyi

判断：只有“模型固定、流量巨大、值得为每微秒优化”的团队才该学这招。通用框架选 Python 是对的。

SCHEDULER · STATE

三个队列，一条状态机



- **decode 步必须清空 running**：每个 seq 要么被调度、要么被抢占，`TORCH_CHECK(l == r)`。
- **被抢占的请求**：`reset_for_recompute()` 后 `push_front(waiting)`，下一轮重新 prefill。
- **锁**：`BlockManagerMutexAcquire` 同时锁 block 和 state 两个管理器。

NORMAL 引擎：两轮 allreduce 决定做不做 prefill

ALL 点

全员都想做才做

每 prefill_attempt_interval 步。防“一个 rank 干活、其他 rank 陪跑空 prompt”。

allreduce(want?1:0, MIN) == 1

ANY 点

任一 rank 想做就做

每 prefill_any_attempt_interval 步。给孤立 rank 兜底，防饿死。

allreduce(want?0:1, MIN) == 0

```
// 第二轮：want 只看 waiting 非空，_schedule_prefill 仍可能空手
// 全组都空手必须转 decode，否则 decode_step 被清零 → 下一步再投票 → 死循环
bool any_scheduled = (_dp_allreduce(outputs ? 0 : 1, MIN) == 0);
if (!any_scheduled) return _schedule_decode(acquire);
```

chunked prefill 的五维裁剪

维度	变量	作用
单条上限	`prompt_limit`	不开 chunked = min(max_model_len, max_num_batched_tokens)
token 预算	`max_num_batched_tokens`	所有 seqlen_q 之和
KV 总长	`prefill_max_sum_seqlen_k`	防长上下文把 attention 的 K 维和算爆
并发数	`max_num_seqs`	—
块/slot	`_can_allocate_with_state`	返回 LATER 就 break，等下一轮

切分还要对齐：CP 的 `cp_size*2`、不切进 image pad、支持 `chunk_checkpoints` 强制断点。

LCFS + 抢占：尾延迟优先

按 output_len 降序排 → 从最短的开始调度 → 块不够？从最长的抢

抢占后：重算

`reset_for_recompute()`，回到 waiting 队头。适用于普通请求。

抢占后：结束

`stop_preempted_request=true` 或请求带 SO 状态（自动机不能重算）。

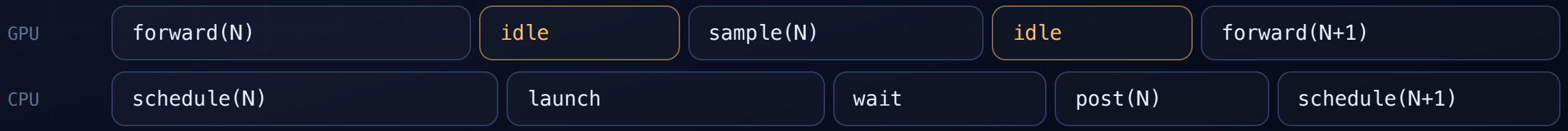
为什么 LCFS：让短请求尽快完成，降低平均排队；长输出本来就要跑很久，抢它重算的边际成本更可控（推断）。

调试先看这 6 条断言

1. `sum(seqlens_q) == num_batched_tokens <= max_num_batched_tokens`
2. `seqlen_q + cache.num_tokens <= seq.get_len()`
3. `position_end <= runtime_max_model_len`
4. decode 步 running 必须清空（全部调度或抢占）
5. COMPLETION 必须 `stop_preempted_request=true`
6. `nextn>0` 时，MTP info slot 使用数 == running 大小

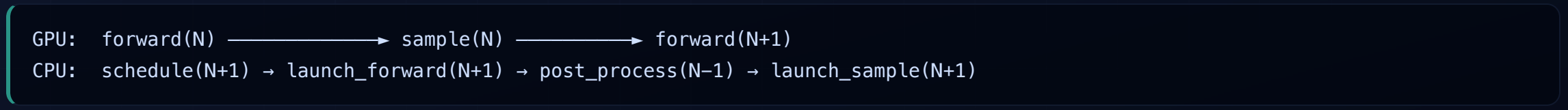
这些不是“注释里的建议”，是 TORCH_CHECK。线上撞到任何一条，先回看是哪一步的调度状态被破坏了。

decode 一步里，GPU 有两段空转



- `post_process(N)` 必须先拿到 sample 的 GPU→CPU 同步结果。
- decode 的 step 是毫秒级，CPU 这两段可能占 5%–20%；batch 越小占比越高。
- ZOS 的目标不是“调度更快”，而是“调度时 GPU 不空转”。

把 CPU 提前一步



1
forward(N)
当前步主模型

2
post(N−1)
补上一步的后处理

3
sample(N)
当前步采样

4
延迟 post(N)
留给下一步 forward 重叠

顺序不能换成 1→3→2：那样 post(N−1) 要等 sample 同步，白等。

三个新问题，三套补丁

资源晚释放

`DeferredRelease`：本步产出挂起，下一步 `pre_sampler_sync` 才释放。

空 rank 也要动

`placeholder step`：跑一个假 forward，保持 DP 组步型一致、NCCL 不超时。

长度未知

CPU 乐观估计 `seqlens_q`；GPU 用 D2H 回来的真实值修正 metadata。

代价：输出“晚一步”、调试难、资源生命周期变长；收益：稳态下 GPU 的 forward→sample→forward 没有 gap。

为什么是两套缓存

维度	PAGED BLOCK KV	STATE SLOT
ID	`block_id`	`state_id`
粒度	block_size token	每序列一个固定 slot
存什么	主 KV + indexer KV	SWA KV + 压缩状态
何时写	每步	只在 128 token 边界
进前缀缓存	是（外部 KV store）	否（只在产生它的机器）
容量	`num_gpu_blocks`	`num_state_seqs`

生命周期、访问模式、介质、命中策略都不同，硬塞进一个 pool 会把逻辑搅烂。

前缀缓存不在引擎里

Entrypoint
哈希链前缀匹配（只查 metadata）

——hit_tokens——▶

3FS KV Store
异步拉到 host → GDR swap-in

——▶

Scheduler
只分配命中之后的 block

引擎内有什么

- 极简 free list + refcount
- fork 请求的 pin + COW
- 没有 hash / LRU / radix tree

因此不能假设

- ✕ 同前缀自动复用
- ✕ `hit_tokens > 0` 就代表数据已加载
- ✕ 单机能复现生产命中率

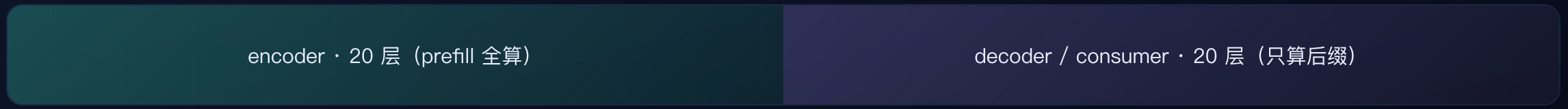
KV CACHE · PRECISION

一份权重，四种精度

内容	格式	SCALE 粒度	为什么
主 KV	FP4 e4m3	per-16	最敏感，要更细的 scale
Indexer KV	MXFP4 ue8m0	per-32	只用于打分选块，粗一点没关系
MoE 权重	MXFP4 ue8m0	per-32	专家多、访问稀疏
SWA KV	MXFP8	per-32	只有 20 层、134 token，量小

代码：`derive_main_kv_cache_num_per_channels()`；SWA 单序列约 1.35 MiB（20×134×528 B）。

prefill 只跑前一半层



- **CED**：decoder 的 global KV 由 encoder 最后一层 hidden 投影而来，不逐层自算。
- **代码**：`cfg.n_prefill_layer` 之后是 consumer；forward 在边界层切 metadata 视图。
- **激活参数**：外部资料称 prefill 8B / decode 16B；代码只认配置，不写死。

20/20 为外部资料，代码只认 `n_prefill_layer`。这是“模型结构直接长进推理引擎”的典型：省的不是调度，是整段计算。

mask 与 compact：一个改可见域，一个真删行

MASK

不真截断

- 仍算完整 token 行
- 把矩形外的 SWA 可见域置空（整行 −1）
- 实现简单，几乎不省算力

COMPACT

真的只留后缀

- gather 出保留行，组成更短 varlen
- hidden / residual / topk / candidates 一起搬
- 约省一半 prefill 计算

两者共用同一个 SWA 可见域公式，所以在保留区间内数值一致 —— 这是 `examples/yoco/` 要验证的。

重算 128 个 token，而不是 2,560 个



1. 命中上限 = L − 134（128 窗口 + 6 回溯）
2. 先查 block KV → L1（128 的倍数）
3. 再查 [L1−128, L1] 的 SWA：有则命中 L1
4. 没有则退一个 block 到 L1−128，重放补齐

近似结果只用于 decode，不进入 prefix cache，避免误差累积。

一个 attention 类，四种机制

压缩 KV（C1/C2）

gated low-rank 压缩；只允许 ratio ∈ {0,1,2}，C4/C128 已删除。

稀疏索引（MQA）

`relu(q@k)·w` 打分 → top-k 选块；单头，不用 softmax。

SWA

局部窗口兜底；与稀疏选中的全局块拼接。

跨层共享

Full / Reindex / Reuse：共享 main KV、indexer K 或 top-k 索引。

约束：稀疏必须配 SWA；不开 SWA 必须有 packed KV；indexer 头数必须全局唯一。

Full / Reindex / Reuse：谁算 KV，谁算索引

模式	MAIN KV	INDEXER K	TOP-K 索引	代码线索
Full	自己算	自己算	自己选	`is_kv_source`
Reindex	复用	自己算	自己选	`is_index_source`
Reuse	复用	复用	复用	`shared_topology`

- 共享后，后面几十层只读同一份 main KV，只维护很小的 indexer K。
- 分层索引器让后段只在“第一个 Full 层给出的候选池”里选，成本与上下文长度解耦。

逻辑专家 → 物理副本 → 卡

logical · 384
模型定义的专家

——复制——▶

physical · 448
多放 64 个热门副本

——布局——▶

rank × 7 槽
EP64，每卡 7 个

映射表

`physical_to_logical` · `logical_to_physical`，每层一份。
dispatch / 路由 / sampler 直接读。

指标

`moe_lb` = 最慢 rank / 平均。1.09 = 最慢比平均多 9% 的活。

EPLB · ALGORITHM

两步：先决定复制谁，再决定放哪张卡

STEP 1

冗余给谁

每轮给一个专家加副本，选“加完后最大每副本负载最低”的那个。

```
score = tokens / logical_count
score_added = tokens / (count+1)
argmin( max(score with one replaced) )
```

STEP 2

放哪张卡

逐层贪心：最小化各 step 的 rank 峰值之和。

```
minimize  Σ_step  max_rank(load)
```

搬权重走 NVLink symm_mem 原地交换；decode 1 万步重排一次，prefill 1000 步。

固定验证 5 个，是浪费还是保险?

固定长度

每条请求都验证 5 个 draft。实现简单、可预测；但低概率 token 也在花验证算力。



`verify = 5.0`

DSpark 变长

confidence head 预测每个 draft 的接受率，按期望收益排序，选前缀子集。



`verify = 3.2 – 4.4`

平均接受长度几乎不变（3.29 → 3.20），验证算力少 ~30%。

两个队列，两个时间尺度

N−2 轮

决定总预算 best_bsz

总量错误代价大，用更旧、更稳的接受率。

N−1 轮

分配每请求长度

相对新，用于给高接受率的 draft 加格子。

```
throughput(bsz) = expected_tokens / (baseline_ms + slope_ms × graph_size + sampler_ms)
expected_tokens = num_seqs + Σ accept_rate(draft)
```

按“单个 draft token”分配，不是按请求；一个请求可能拿到 0–5 个。ZOS 下这一步搬到 GPU（cumprod + global rank + remap）。

收益随并发递增，低并发会亏

并发（SEQ/RANK）	FIXED-5 验证长度	DYNAMIC 验证长度	接受长度	吞吐增益
16	5	4.44	3.24	−1.5%
32	5	3.65	3.17	+1.6%
48	5	3.94	3.26	+3.6%
60	5	3.58	3.21	+5.4%
72	5	3.24	3.12	+6.7%
96	5	3.24	3.15	+11.0%

数据来源：仓库 Q&A，EP64/DP64，nextn=5。低并发固定开销占比高，少验证省不了多少。

SAMPLER

三件事：融合、可复现、可约束

Fused

温度、top-k/p、log-softmax、采样、SO mask 一个 kernel 完成，避免 logits 反复读写。

Batch-invariant

固定 reduction 顺序；不同 batch 组成下 bit 级一致（NVIDIA）。

Deterministic

hash 随机数 + 固定 `seq_seed` + 关 MTP varlen；相同输入相同输出。

```
# fused_sample.py — 一个 kernel 里做完五步
temperature → top-k/top-p → log-softmax → sample → (optional) logprobs / SO mask
```

没有确定性，`tests/models/offline_decode_ppl.json` 的逐样本 PPL 金标准就无法比对。

STRUCTURED OUTPUT

编译、匹配、掩码：三段解耦



- **异步编译**：请求先提交，编译完由 `pump()` 收割并入调度；编不完不阻塞引擎。
- **失败隔离**：编译失败/超时只杀该请求（`FINISHED_SO_FAILED`）。
- **投机回滚**：draft 被拒时 `rollback(steps)`，否则自动机状态错位。

DUAL HARDWARE

一套 Python，两套 kernel

Platform 抽象

- `tilelang_ops_backend`：cuda / asc
- `distributed_backend`：nccl / hccl
- `fp8_sf_type`：scale 类型
- `pin_host_memory`：异步 H2D

双份算子

`tilelang_ops/cuda/` 30 个文件

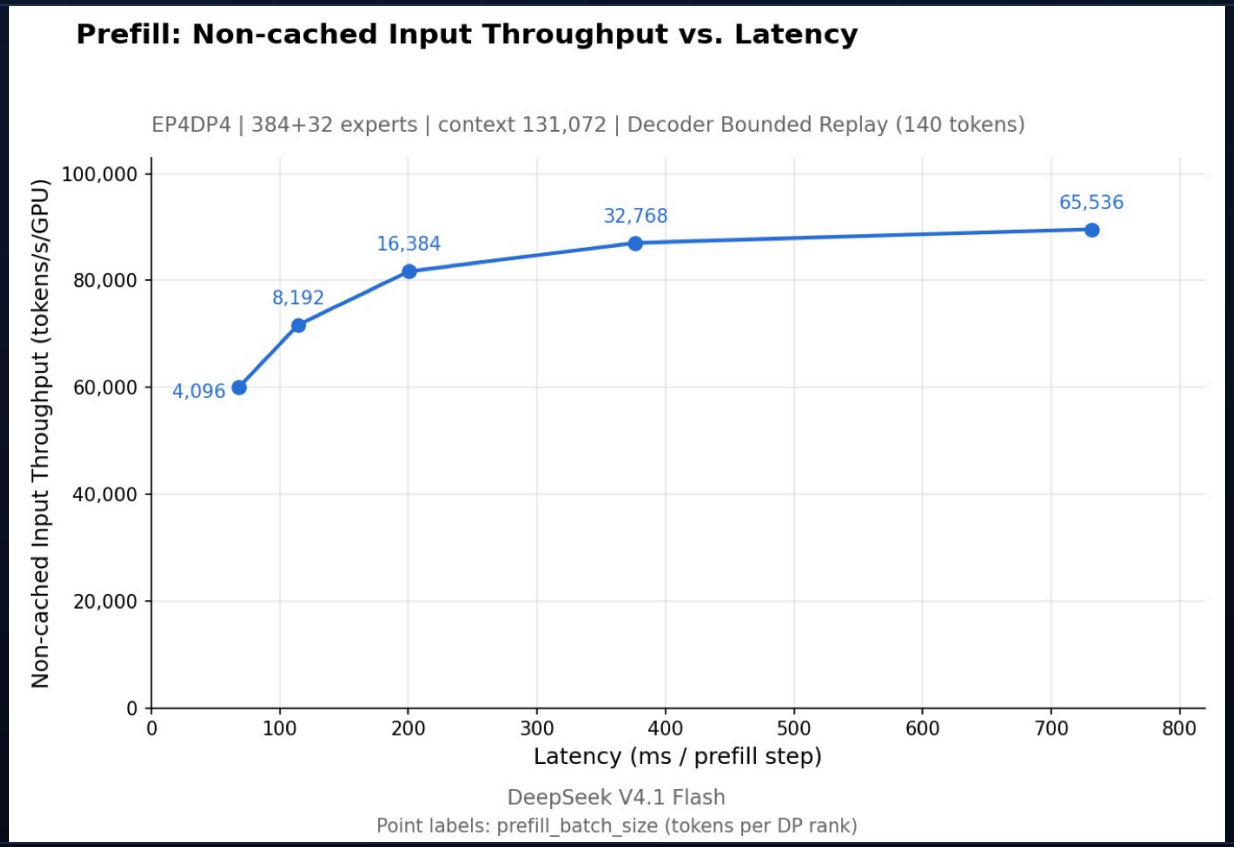
`tilelang_ops/asc/` 30 个同名文件

约 1.1 万行重复实现，换每平台的极致性能。

构建：Python 依赖 → vendor wheel（DeepGEMM/FlashMLA/DeepEP/DeepSelect）→ _C 扩展 → 运行时 TileLang JIT。

PERFORMANCE · PREFILL

Prefill：2 万 token 后曲线饱和

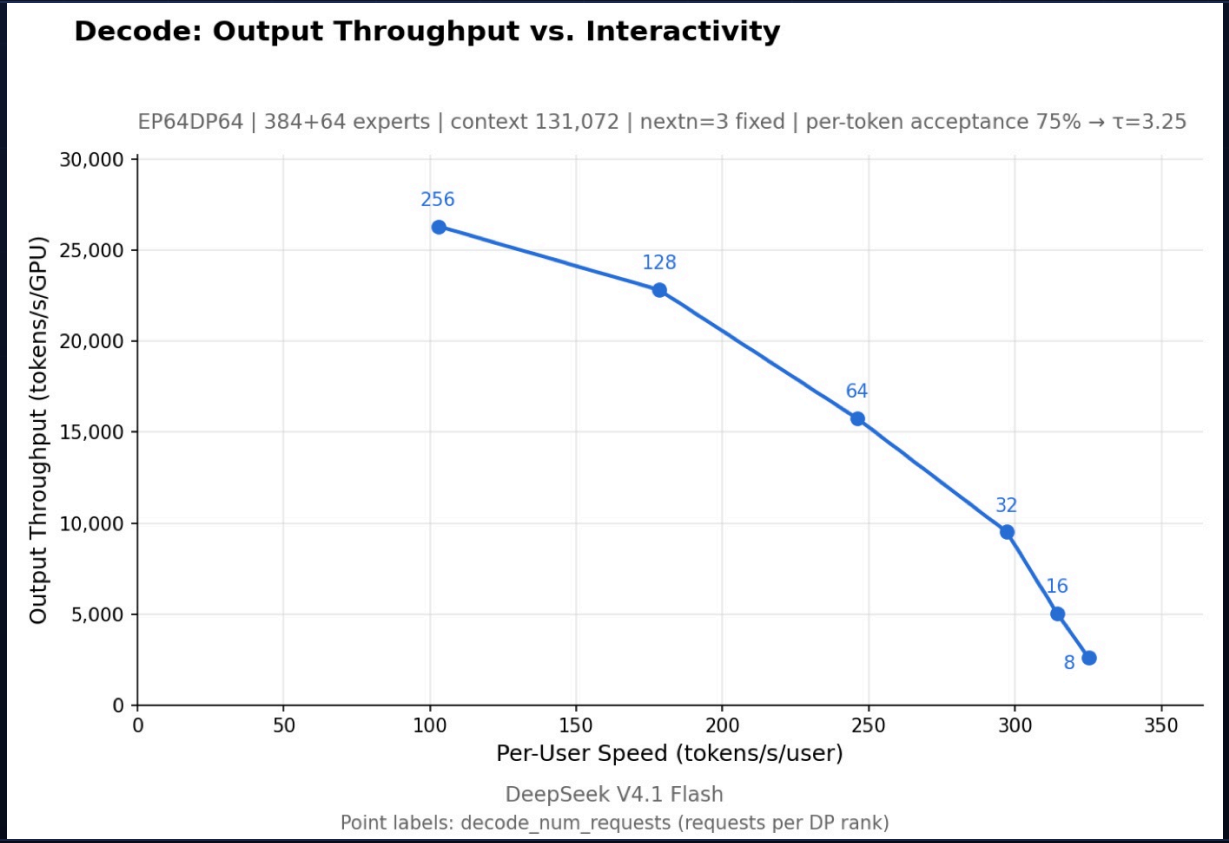


4k → 60k tok/s/GPU，约 70 ms　**32k → 86k** tok/s/GPU，约 370 ms　**65k → 88k** ：收益主要来自 YoCo，不是堆 batch

EP4DP4 · 131k ctx · bounded replay 140 · 384+32 experts

PERFORMANCE · DECODE

Decode：吞吐与交互性的硬 trade-off

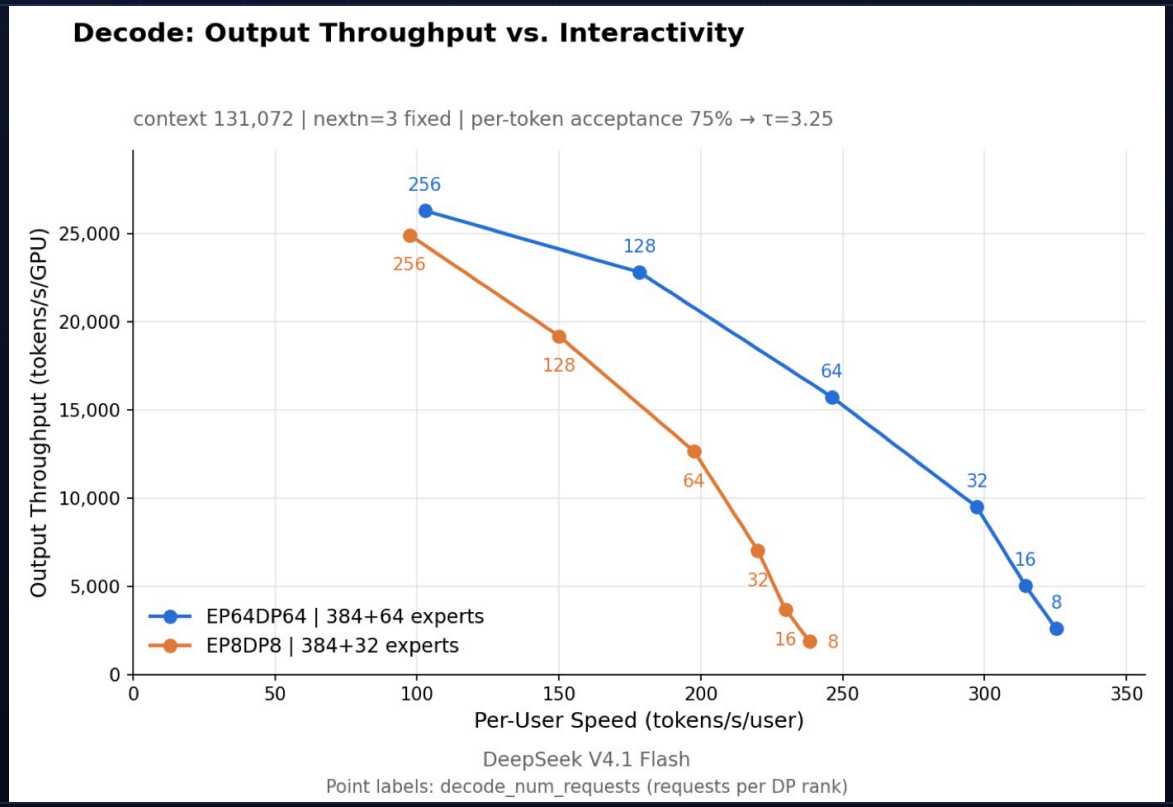


64 并发 ：15.5k tok/s/GPU，249 tok/s/user **256 并发** ：26k tok/s/GPU，但只剩 104 tok/s/user **生产点** 按 SLA 选，不是无脑堆并发

EP64DP64 · 131k ctx · nextn=3 · 75% 接受 · 384+64 experts

PERFORMANCE · WIDE-EP

为什么 decode 要 EP64 而不是 EP8



175 tok/s/user ： EP8 约 15k → EP64 约 22.5k（1.5×）　**250 tok/s/user** ： EP8 约 8k → EP64 约 16k（2.0×）

EP 越大，单步 MoE 每卡访存量越小；配合 EPLB，把最慢 rank 压到平均的 1.09 倍。

COMPARISON

和主流框架的结构性差异

维度	DEEPINFER	VLLM V1	SGLANG	TENSORRT-LLM
模型	仅 V4.1-Flash	极广	很广	广（需编译）
硬件	SM100 / Ascend 950	NVIDIA/AMD/TPU/CPU	NVIDIA/AMD	NVIDIA
并行	EP+DP+CP，无 TP	TP/PP/DP/EP	TP/DP/EP/PP	TP/PP/EP
调度器	C++	Python	Python + overlap	C++ runtime
前缀缓存	引擎外（3FS）	自动 block hash	RadixAttention	需配置
投机	DSpark 变长	固定为主	EAGLE/draft	Medusa/EAGLE
在线服务	无	OpenAI server	OpenAI/gRPC	Triton/server

竞品为 2026 年公开资料的共性描述；DeepInfer 为代码实证。

COMPARISON · FIVE SHIFTS

五个“选择”，不是五个“优化”

C++	调度器换语言：用可维护性换 decode 小 step 的确定性开销。
3FS	前缀缓存外移：用系统复杂度换任意前缀命中与存储/计算解耦。
EP	放弃 TP：MoE 只传 token 不传权重，配合 EPLB 摊薄每卡访存。
DSpark	投机从固定长度变成按边际收益分配验证算力。
单模型	一个模型榨干两代硬件，而不是一个引擎适配 N 个模型。

SELECTION

什么时候选谁

DeepInfer

部署 V4.1-Flash，手里有 SM100 / Ascend 950；要吃到 YoCo + CSA2 + FP4 KV 的全部收益。

vLLM / SGLang

部署任意 HF 模型，要 OpenAI API、LoRA、多租户、广泛硬件。

TensorRT-LLM

NVIDIA 上极致低延迟，能接受模型编译与较重的工程约束。

LMDeploy

快速起一个中文社区友好的 serving，工具链完整。

读 DeepInfer 是为了学设计；选型时不要因为它“新”而忽略它的边界。

LEARNING PATH

四周：从骨架到能改

WEEK 1

骨架

- 01 全景
- 02 生命周期
- 03 调度器
- 产出：时序图

WEEK 2

调度与缓存

- 04 ZOS
- 05 两套缓存
- 08 EPLB
- 产出：不变量清单

WEEK 3

模型与 kernel

- 06 YoCo
- 07 CSA2
- 09 DSpark
- 产出：分支树

WEEK 4

输出

- 13 实验
- 14 对比
- 专题笔记
- 产出：10 页 slides

有 GPU 走“实验派”路线（先 13 再 04/06/09）；没有 GPU 走“源码读者”路线。

SELF-TEST

能答上这 5 题，就算读懂了

Q1 为什么 NORMAL 引擎需要**两轮** allreduce？第二轮防的是什么死循环？

Q2 ZOS 下为什么 `post_process(N-1)` 必须放在 forward(N) 之后、sample(N) 之前？

Q3 YoCo 命中策略里，134 从哪来？为什么重放结果不能进 prefix cache？

Q4 EPLB 为什么按“多 step 峰值之和”排布，而不是平均负载？

Q5 DSpark 为什么用 N-2 轮接受率选总预算、N-1 轮分配每请求长度？

答案分别在 docs/03、04、06、08、09。

PITFALLS

八个最容易踩的坑

1. vendor/ 是 submodule，普通 clone 是空目录。
2. H100/A100 跑不了：只认 SM10。
3. 没有在线 server，LLM 只是离线批量。
4. `hit_tokens` ≠ 数据已加载。
5. 自己写循环忘了 `pump()`，SO 请求永远不入调度。
6. ZOS 下加 `.item()` 会触发同步守卫。
7. 改 `block_size` 不只是改配置，kernel/CP/128 边界都耦合。
8. 低并发开 MTP varlen 可能负收益。

ONE PAGE

一页纸总结

定位	DeepSeek 生产框架的开源子集：离线、单模型、两代硬件。
结构	C++ 调度器 + Python 编排 + TileLang kernel + EP/DP，无 TP。
三招牌	ZOS 流水线、YoCo bounded replay、DSpark 变长验证。
最大差异	前缀缓存在引擎外（3FS），引擎内只有 fork/COW。
性能	prefill 60–88k；decode 2.5–26k tok/s/GPU（交互性 325–104）。
学习价值	看“通用框架如何为单一模型做外科手术”。

FIN

读它，不是为了用它。

是为了知道“快”可以被拆成多少个小决定。

```
repo    ~/DeepInfer
docs    ~/deepinfer-study/docs/
slides  ~/deepinfer-study/slides/DeepInfer-深度解析.html
```

下一步：挑一个专题（ZOS / YoCo / DSpark），读到能改，再设计一个验证实验。